

Unit V: Object-Oriented Programming (OOP)

Section 1: Introduction to OOP

Python supports **both structured (procedural) programming** and **object-oriented programming (OOP)**.

- **Structured programming:** Focuses on functions and procedures.
 - **OOP:** Focuses on **objects** that combine data and behavior, making programs **modular, reusable, and easier to maintain**.
-

1.1 Features of Object-Oriented Programming

Feature	Description	Example
Class	Blueprint for objects	<code>class Student: pass</code>
Object	Instance of a class	<code>s1 = Student()</code>
Methods & Message Passing	Functions inside a class that operate on object data	<code>s1.get_grade()</code>
Inheritance	Child class inherits properties of parent class	<code>class GradStudent(Student): pass</code>
Polymorphism	Same interface behaves differently	<code>print(len("abc")), print(len([1, 2, 3]))</code>
Containership	One object contains another object	<code>class Engine inside class Car</code>
Reusability	Code can be reused through inheritance	Use base class methods in child class
Delegation	Object delegates tasks to another object	Object calls methods of contained objects
Data Abstraction	Hide unnecessary details from user	Public methods interact with private data
Encapsulation	Restrict direct access to object data	Private variables: <code>__var</code>

Section 2: Classes and Objects

2.1 Classes

- A **class** is a blueprint for creating objects.
- **Syntax:**

```
class ClassName:  
    # attributes  
    # methods
```

2.2 Objects

- An **object** is an **instance of a class**.

```
class Student:  
    def __init__(self, name, age):  
        self.name = name # object variable  
        self.age = age  
  
s1 = Student("Alice", 20)  
print(s1.name) # Output: Alice
```

2.3 Class Method and self Argument

- **self** → Represents the **instance of the class**. Used to access object variables.

```
class Student:  
    def greet(self):  
        print("Hello", self.name)  
  
s1 = Student()  
s1.name = "Alice"  
s1.greet() # Output: Hello Alice
```

2.4 __init__() Method

- **Constructor method** executed when object is created.
- Initializes object variables.

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
s1 = Student("Bob", 21)  
print(s1.age) # Output: 21
```

2.5 Class Variables and Object Variables

- **Class Variables:** Shared among all objects.
- **Object Variables:** Unique to each object.

```
class Student:
    school = "XYZ School" # class variable

    def __init__(self, name):
        self.name = name # object variable

s1 = Student("Alice")
s2 = Student("Bob")
print(s1.school, s2.school) # Both share class variable
```

2.6 __del__() Method

- **Destructor method** called when object is destroyed.

```
class Student:
    def __del__(self):
        print("Object destroyed")

s1 = Student()
del s1 # Output: Object destroyed
```

2.7 Public and Private Members

- **Public members:** Accessible outside class.
- **Private members:** Accessible only inside class. Prefix with `__`

```
class Student:
    def __init__(self):
        self.__id = 101 # private
        self.name = "Alice" # public

s = Student()
print(s.name) # OK
# print(s.__id) # Error: Private member
```

2.8 Built-in Functions to Check, Get, Set, Delete Class Attribute

Function	Description
<code>hasattr(obj, "attr")</code>	Check if attribute exists
<code>getattr(obj, "attr")</code>	Get attribute value
<code>setattr(obj, "attr", value)</code>	Set attribute value

Function	Description
<code>delattr(obj, "attr")</code>	Delete attribute

Example:

```
class Student:
    def __init__(self):
        self.name = "Alice"

s = Student()
print(hasattr(s, "name")) # True
print(getattr(s, "name")) # Alice
setattr(s, "name", "Bob")
print(s.name)             # Bob
delattr(s, "name")
```

2.9 Garbage Collection

- Python automatically **freeds memory** when objects are no longer in use.

```
import gc
print(gc.isenabled()) # True if garbage collection is enabled
```

2.10 Class Methods and Static Methods

- **Class Method:** Works on class variables, not object variables. Decorate with `@classmethod`.

```
class Student:
    school = "XYZ School"

    @classmethod
    def get_school(cls):
        return cls.school

print(Student.get_school()) # Output: XYZ School
```

- **Static Method:** Does not access class or object variables. Decorate with `@staticmethod`.

```
class Student:
    @staticmethod
    def info():
        print("This is a student class")

Student.info() # Output: This is a student class
```

Key Takeaways

- OOP allows **modular, reusable, and organized code**.
- **Classes** define objects; **methods** define behavior.
- **Encapsulation and data abstraction** help **protect and hide data**.
- **Inheritance, polymorphism, and containership** promote code **reusability and flexibility**.
- Python provides **built-in support** for constructors, destructors, class methods, static methods, and garbage collection.